

Doppler Ultrasound Segmentation and Metric Extraction Pipeline

Dale Kernot

July 28, 2026

Contents

1	Introduction	3
1.1	Motivation and clinical context	3
1.2	Scope of this document	3
1.3	Document conventions	3
1.4	Codebase overview	3
2	Architecture and Entry Points	5
2.1	Single-image entry point	5
2.2	Batch entry point	5
2.3	Configuration	5
2.4	Environment prerequisites	5
3	Input Classification and Modality Dispatch	6
3.1	Extension-based dispatch	6
3.2	Why the two branches diverge	6
4	Stage 1: Text Extraction and OCR Repair	7
4.1	Metric panel isolation: <code>colour_extract_vectorized</code>	7
4.2	OCR and line assembly: <code>text_from_greyscale</code>	7
4.3	Target label sets and vessel families	8
4.4	Label matching	8
4.5	Value parsing and numeric repair	9
4.6	Heart rate refinement	9
4.7	Metric consistency checks	9
5	Stage 2: Coarse Localisation of the Doppler Region	11
5.1	Raster: <code>initial_segmentation</code>	11
5.2	Raster: <code>define_end_rois</code>	11
5.3	DICOM: region bounds from metadata	11
6	Stage 3: Axis Calibration	12
6.1	Raster: tick detection with <code>search_for_ticks</code>	12
6.2	Raster: label detection and pairing with <code>search_for_labels</code>	13
6.3	Raster: validation of ticks	13
6.4	Raster: cross-checking the two axes	14
6.5	Raster: zero-line estimation	14
6.6	DICOM: calibration from metadata	14
6.7	Tolerance of missing axes	15

7	Stage 4: Waveform Segmentation and the Three Tracing Methods	16
7.1	Orchestration: <code>segment_refinement</code>	16
7.2	The shared refined mask: <code>refine_waveform_segmentation</code>	16
7.3	Envelope orientation: upper or lower	17
7.4	Method 1: Morphological envelope	17
7.5	Method 2: Ray tracing	17
7.6	Method 3: Constrained region growing	18
7.7	Dispatch and comparison: <code>compute_top_curve</code>	19
7.8	Debugging aids	19
8	Stage 5: Digitisation and Calibration	20
8.1	Condensing the curve to one sample per column	20
8.2	Raster: choosing an axis and mapping rows to velocities	20
8.3	Raster: the arbitrary time axis	20
8.4	Raster: time-axis calibration from heart rate	21
8.5	DICOM: mapping pixels using physical deltas	21
8.6	Shape-preserving smoothing	22
8.7	Primary and overlay series: an asymmetry between entry points	22
9	Stage 6: Beat Detection and Haemodynamic Indices	23
9.1	Why feet rather than troughs delimit a beat	23
9.2	Peak proposal	23
9.3	Foot detection	23
9.4	Per-beat extrema	24
9.5	Signal quality filtering	24
9.6	Index formulae	24
10	Stage 7: Model Comparison and Selection	26
10.1	Mapping digitised values into the metric table	26
10.2	Agreement scoring	26
10.3	Selecting the returned model	27
10.4	DICOM: no independent ground truth	27
11	Batch Pipeline and Outputs	28
11.1	File discovery: <code>get_likely_us</code>	28
11.2	Per-scan processing: <code>segment</code>	28
11.3	Rendered outputs	29
11.4	Serialised artifacts	29
11.5	HTML report	29
12	Error Handling and Known Failure Modes	31
12.1	Degradation strategy	31
12.2	Commonly observed failures	31
12.3	Limitations	31
13	Implementation Notes	33
13.1	Performance considerations	33
13.2	Testing	33
13.3	Diagnostics and figure conventions	33
14	Recommended Follow-Up	34

1 Introduction

1.1 Motivation and clinical context

Doppler ultrasound scans contain two linked sources of information. The first is the image of the Doppler waveform, which can be segmented and digitised into a velocity-time trace. The second is the machine-generated text panel, which commonly reports clinical measurements such as peak systolic velocity, end diastolic velocity, heart rate, pulsatility index, and resistive index.

The aim of this pipeline is to extract both sources of information from a scan, compare them where possible, and return a structured table of measurements together with a digitised waveform trace. The code is designed around scans with a clearly visible Doppler waveform and machine annotation, but the implementation contains fallback behaviour for incomplete axes, missing labels, and scans where only part of the expected visual structure is present.

Two fundamentally different classes of input are supported, and they are calibrated in fundamentally different ways. Screenshot-style raster images (JPEG, PNG and similar) carry no calibration metadata, so the pipeline must recover the velocity scale by reading the axis tick marks and numeric labels off the image itself. DICOM files carry explicit spatial calibration in their metadata, so the pixel-to-physical-unit mapping is read directly rather than inferred. Keeping these two paths clearly separated is essential to understanding the codebase, and this document treats them as distinct branches throughout.

1.2 Scope of this document

This document describes the processing pipeline implemented in the `usseg` package. It is organised by processing stage. For each stage it states what the stage is trying to achieve, which functions implement it, what data structures flow in and out, and how the raster and DICOM paths differ.

The single-image entry point, `data_from_image`, is used as the reference implementation of the core algorithm because it contains the algorithmic path with the least surrounding bookkeeping. The batch pipeline performs the same algorithmic stages and adds file discovery, output writing, annotation rendering, serialisation and report generation.

1.3 Document conventions

Function and module names are written in a monospaced font, for example `segment_refinement`. Unless stated otherwise, functions named without a module prefix live in `general_functions.py`, which holds the majority of the algorithmic code.

Image coordinates follow the convention used in the code: arrays are indexed as `[row, column]`, equivalently `[y, x]`, with the row index increasing downwards from the top of the image. Consequently a numerically smaller row index is visually higher in the image, which matters when reading the envelope-orientation logic in Section 7.

1.4 Codebase overview

The modules involved in the processing pipeline are:

`single_image_processing.py` Provides `data_from_image`, the single-scan API used by examples and tests.

`segment_files.py` Provides `segment`, the batch workflow over a list of scan files, including annotated and digitised output rendering and serialisation.

`general_functions.py` Contains the majority of the implementation: OCR and text repair, coarse and refined segmentation, the three waveform tracing methods, axis detection, digitisation, beat detection, metric calculation and annotation.

`hemodynamic_indices.py` Pure functions for the haemodynamic indices (pulsatility index, resistive index, time-averaged maximum velocity).

`digitized_comparison.py` Compares digitised measurements against OCR-derived measurements and selects the best-matching segmentation model.

`organise_files.py` Discovers candidate ultrasound files within a configured directory tree.

`visualisation.html.py` Builds the HTML report used to inspect batch results.

`setup_environment.py` Configures the Tesseract OCR environment.

`main.py` Batch entry point that reads `config.toml`, discovers files, segments them and generates the HTML report.

2 Architecture and Entry Points

2.1 Single-image entry point

`data_from_image` in `single_image_processing.py` processes exactly one scan and returns its results in memory without writing any files. It is the recommended way to inspect the behaviour of the algorithm on a single input.

```
from usseg import data_from_image

df, (xdata, ydata) = data_from_image(image_path="path/to/scan.png")
```

The function returns two objects:

df A pandas dataframe. For raster images this contains the OCR-extracted text metrics, the digitised equivalents of those metrics for each segmentation model, comparison fields, and a **Returned model** entry recording which model was selected. For DICOM input it contains metrics derived purely from the digitised waveform, optionally preceded by a vessel/side label row.

[xdata, ydata] The digitised waveform trace. For raster images this is the trace of the selected model after heart-rate time scaling. For DICOM it is the primary digitised series in physical units.

A legacy calling convention accepting pre-loaded Pillow and OpenCV image objects is still supported, but it cannot express the DICOM path and is scheduled for removal. Passing `image_path` together with either legacy argument raises **ValueError**.

2.2 Batch entry point

The batch pipeline is driven by `main.py`, which reads `root_dir` from `config.toml` and calls, in order, `setup_tesseract`, `get_likely_us`, `segment` and `generate_html_from_pkl`. Each of these three processing calls is wrapped in the `prof` helper, which profiles the call with `cProfile` and writes cumulative-time statistics to a per-function log file. Running the module directly also configures logging at **INFO** level into `batch_main.log`.

```
python -m usseg.main
```

2.3 Configuration

Configuration is read from `config.toml` in the working directory. The keys are `root_dir`, the directory tree to search for scans; `output_dir`, where rendered PNG outputs are written; and a `[pickle]` table naming the intermediate serialisation files. Of the three pickle keys, `likely_us_images` and `segmented_data` are used; `patient_paths` is declared but never read by any module.

The file is loaded by relative path, so all commands must be run from the repository root.

2.4 Environment prerequisites

Text extraction depends on Tesseract OCR, configured by `setup_tesseract` in `setup_environment.py`. On Windows the executable is expected at `C:/Program Files/Tesseract-OCR/tesseract.exe`. Because every metric read from the image panel and every axis label depends on OCR, an incorrectly configured Tesseract installation causes the text and axis calibration stages to fail even though segmentation may still succeed.

3 Input Classification and Modality Dispatch

3.1 Extension-based dispatch

Both entry points classify each input by file extension and set two mutually exclusive flags, `us_image` and `us_dicom`. Raster extensions are `.jpg`, `.jpeg`, `.png`, `.bmp`, `.gif`, `.tif`, `.tiff` and `.webp`; DICOM extensions are `.dcm` and `.dicom`. Any other extension raises `ValueError` in the single-image path, and is skipped with a warning in the batch path.

Raster inputs are loaded twice, into a Pillow image and an OpenCV BGR array. The Pillow copy is converted to RGB and used for the colour-based operations (text colour isolation and coarse segmentation); the OpenCV array is used for thresholding, contour work, OCR input and all three tracing methods. DICOM inputs are read with `extract_dicom_metadata` and `extract_doppler_from_dicom`, which return the calibration metadata and an image pair cropped to the Doppler region.

3.2 Why the two branches diverge

The single most important structural difference is the source of calibration. Table 1 summarises how the stages differ.

Stage	Raster image	DICOM
Metric text	OCR of the on-screen panel	Not attempted
Region bounds	<code>initial_segmentation</code>	Region metadata tags
Velocity scale	Detected axis ticks/labels	<code>PhysicalDeltaY</code>
Time scale	Inferred from heart rate	<code>PhysicalDeltaX</code>
Zero line	<code>estimate_zero_line_y</code>	<code>ReferencePixelY0</code>
Yellow exclusion	Enabled	Disabled
Output metrics	OCR compared with digitised	Digitised only

Table 1: Stage-by-stage differences between the two processing branches.

Everything from Section 7 onwards, that is the refined mask and all three tracing methods, is shared between the branches. The branches differ before segmentation, in how the region of interest and the calibration are established, and after segmentation, in how pixel coordinates are converted into physical units and which metrics are reported.

4 Stage 1: Text Extraction and OCR Repair

Raster images only. This stage is skipped entirely for DICOM input, where the metric panel is generally absent from the extracted Doppler region.

The purpose of this stage is to recover the clinical measurements that the ultrasound machine has already printed on the screen. These values serve two roles: they are reported as the primary metric table, and they act as the reference against which the digitised waveform is later scored, which is what ultimately selects one of the three segmentation models.

4.1 Metric panel isolation: `colour_extract_vectorized`

On the supported machines the metric panel is rendered in a distinctive yellow. Rather than thresholding brightness, the code isolates that colour geometrically: the target colour defines the axis of a cylinder in RGB space, and a pixel is retained if it falls inside that cylinder.

The target is `[255, 255, 100]` with both `cyl_length` and `cyl_radius` set to 95. The axis direction is obtained from `append_spherical_1point`, which expresses the target colour in spherical coordinates, and the cylinder is centred on the target with endpoints `start` and `end`. A pixel p is inside when it lies between the two end planes and within the radius of the axis:

$$(p - \text{start}) \cdot v \geq 0, \quad (p - \text{end}) \cdot v \leq 0, \quad \|(p - \text{start}) \times v\| \leq \text{cyl_radius} \|v\|,$$

where $v = \text{end} - \text{start}$. Matching pixels are set to white and all others to black, producing a binary mask image. Because the test is a distance in colour space rather than a per-channel range, it tolerates the brightness variation of anti-aliased text.

An earlier non-vectorised implementation, `colour_extract`, remains in the module but is no longer used.

4.2 OCR and line assembly: `text_from_greyscale`

This function performs OCR on the isolated panel and turns the result into a dataframe with columns `Line`, `Word`, `Value` and `Unit`. It also returns a `Fail` flag.

Spatial pre-masking. Before OCR, the mask is cropped by blanking the left two-thirds of the width and the bottom 55 % of the height, because on the supported layout the metric panel occupies the upper right of the screen. This substantially reduces spurious detections from the waveform area.

Primary OCR pass. Tesseract is invoked through `image_to_data` rather than `image_to_string`, so that per-word bounding boxes and confidences are available for the line-grouping step:

```
--oem 1 --psm 3 -c tesseract_char_blacklist=1,!_!=$
```

`oem 1` selects the LSTM engine and `psm 3` fully automatic page segmentation. The character blacklist suppresses glyphs that are commonly confused with digits or that never appear in a metric label.

Line grouping. Words must be reassembled into lines before a label can be matched. The vertical centre of each word box is computed and clustered with DBSCAN over that single coordinate, using `eps = 5` pixels and `min_samples = 2`. Words within a cluster are sorted

by horizontal position and joined with spaces. A consequence of `min_samples = 2` is that an isolated single-word line forms no cluster and is therefore dropped from this source. To compensate, a second pass with `image_to_string` provides newline-delimited lines, and the two sources are combined into a deduplicated union.

The Fail flag. `Fail` is set to 1 when Tesseract returns fewer than 30 text tokens, which is treated as evidence that the image is not a metric-panel scan. It is a heuristic on text quantity, not a per-metric validity flag. Both entry points log a warning and continue when `Fail` is set, but skip `plot_correction`, so a scan flagged this way still yields a segmentation and a trace but no OCR comparison and hence no informed model selection.

4.3 Target label sets and vessel families

Matching is constrained to a vessel family so that, for example, an umbilical label cannot be matched on a uterine scan. The recognised families are held in `METRIC_FAMILY_PREFIXES`, in order:

```
("Lt Ophthalmic", "Rt Ophthalmic", "Lt MCA", "Rt MCA",
 "Lt Ut", "Rt Ut", "Umb", "DV")
```

The order matters because the more specific prefixes are tested first. `_detect_metric_family_from_lines` counts how many OCR lines match each prefix and selects the highest-scoring family; ophthalmic lines additionally increment the count through `_line_looks_ophthalmic`. If no family is detected the full target list is used and a warning is logged.

Vessel	Prefix	Metrics sought
Uterine (left/right)	Lt Ut-, Rt Ut-	PS, ED, S/D, PI, RI, MD, TAm _{ax} , HR
Umbilical	Umb-	as above
Middle cerebral	Lt MCA-, Rt MCA-	as above
Ductus venosus	DV-	S, D, a, TAm _{ax} , S/a, a/S, PI, PLI, PVIV, HR
Ophthalmic	Lt/Rt Ophthalmic A.	PS ($\times 2$), ED, PI, RI, PS/ED, ED/PS

Table 2: Vessel families and the metrics sought for each.

Two label lists exist per family: a plain list and an *extended* list carrying unit suffixes, for example "Lt Ut-PS cm/s". The extended list is used only by the final fuzzy stage described below.

4.4 Label matching

Non-ophthalmic families are matched by a four-stage cascade, each stage operating only on the targets still unmatched by the previous one. This ordering is deliberate: cheap exact tests run first, and progressively looser tests are applied only where necessary.

1. **Exact substring.** The target must appear literally in the line.
2. **Subsequence.** The target's characters, ignoring spaces and hyphens, must appear in order within the line. This absorbs dropped characters without permitting reordering.
3. **Prefix Levenshtein.** The first `len(word)` characters of the line are compared with the target using `rapidfuzz.distance.Levenshtein`, and accepted at an edit distance of 7 or less.
4. **Extended-label distance matrix.** Digits are stripped from the line and the edit distance to each extended label is computed. If the expected metric suffix (for example PS

or PI) appears in the line, the distance is biased by -2 . Each remaining target then takes the line with the smallest non-zero distance, assigned one-to-one so that a line cannot serve two targets.

Ophthalmic matching. Ophthalmic scans bypass the cascade entirely and use `_match_ophthalmic_lines`. The reason is structural: ophthalmic panels report two peak-systolic measurements, so labels cannot be identified by text alone and position must be used. Each line is classified by `_ophthalmic_metric_kind` into PS, ED, PI, RI, PS/ED or ED/PS, and PS lines are assigned to the numbered targets in the order encountered. Orphan numeric lines with no label are attached to the first PS row still holding zero. Laterality is inferred by `_detect_ophthalmic_side_from_ocr`, which counts standalone `Lt` and `Rt` tokens and resolves ties in favour of `Rt`. **This is not a comprehensive solution to Ophthalmic label matching and improvements could be made.**

4.5 Value parsing and numeric repair

Values are taken from the end of the matched line with an anchored regular expression that tolerates OCR-split decimals such as "27 . 35", and captures any trailing non-numeric group as the unit. Spaces are removed from the captured number before conversion. A matched label with no parseable number yields a row with a zero value rather than being dropped, so that the metric's absence is visible downstream.

There is no character-level substitution table (for instance mapping a misread `S` to `5`). Instead, misplaced decimal points are repaired by magnitude: a value that is implausible by an order of magnitude is divided down into its physiological range. `check_pi_value` divides a pulsatility index by 10 in the range 3–10 and by 100 in the range 10–200; the PS/ED normalisation applies the same principle for velocities, dividing by 10 between 250 and 1000 and by 100 between 1000 and 10000; `check_S_D_value` does likewise for ductus venosus values. The approach is effective because the true ranges of these quantities are narrow and well separated from their decimal-shifted counterparts.

4.6 Heart rate refinement

Heart rate deserves special treatment because it is the only quantity that calibrates the raster time axis (Section 8); an error here propagates into every time-dependent result.

After the first pass, `refine_hr_from_local_roi` re-reads the heart rate from a locally enhanced crop. The ROI around the HR line is padded by $\max(15, 35\%)$ of its width and $\max(10, 80\%)$ of its height, then equalised, blurred, Otsu-thresholded and upsampled by a factor of two before a constrained OCR pass using `psm 7` with a character whitelist, falling back to `psm 6`. Candidates are accepted only within 20–240 bpm.

The reconciliation rule is conservative. The refined value replaces the first pass only if the first-pass value is implausible; if both are plausible but differ by more than 3 bpm the first-pass value is kept and a warning is logged. Finally the absolute value is taken, since a spurious minus sign is a common OCR artefact.

4.7 Metric consistency checks

The final step exploits the algebraic relationships between the reported metrics to detect and repair OCR errors. Three variants exist, dispatched by family: `metric_check` for uterine, umbilical and MCA scans, `metric_check_ophthalmic` for ophthalmic, and `metric_check_dv` for

ductus venosus. All of them preserve the original readings in a **Raw Value** column before making corrections, so that repairs remain auditable.

`metric_check` recomputes the three derived quantities from the extracted peak-systolic and end-diastolic velocities,

$$(S/D)_{\text{calc}} = \frac{PS}{ED}, \quad RI_{\text{calc}} = \frac{PS - ED}{PS}, \quad TA_{\text{max,calc}} \approx \frac{PS + 2ED}{3},$$

and compares each with the extracted value at a tolerance of 0.5. Time averaged maximum velocity is treated specially: it is accepted if its magnitude simply lies between ED and PS , because the closed-form estimate above is only an approximation.

The outcome depends on how many of the three agree. If all three agree, nothing is changed. If one or two agree, the disagreeing or missing fields are recomputed from the corrected velocities. If none agree, the code assumes PS is correct and back-solves ED from the reported ratio or resistive index; should both resulting candidates fall below 2, it reverses the assumption, treats ED as correct and recomputes PS . This is the most aggressive repair in the pipeline and the primary reason the raw values are retained.

The ophthalmic and ductus venosus variants follow the same pattern with different tolerances and formulae: ophthalmic uses 0.5 for the ratio and 0.1 for the resistive index, while ductus venosus uses 0.2 for S/a and PI and a looser 2.0 for the approximated time-averaged velocity. The ductus venosus path additionally applies pre-swap heuristics in `text_from_greyscale` to correct values that OCR has attributed to the wrong label.

5 Stage 2: Coarse Localisation of the Doppler Region

5.1 Raster: `initial_segmentation`

`initial_segmentation` finds the approximate extent of the Doppler waveform panel in a raster screenshot. It exploits the fact that the waveform and its surrounding trace are rendered in near-neutral greys, so they can be separated from coloured annotation and from the dark background by a colour-neutrality test rather than by brightness alone.

The function iterates over every pixel of the RGB image and retains the pixel only if the spread across the three channels is small and the pixel is reasonably bright:

$$(\max(r, g, b) - \min(r, g, b)) < 100 \quad \text{and} \quad \max(r, g, b) > 120.$$

Pixels satisfying both conditions are set to white and all others to black. A band at the top of the image is then forced to black to suppress the header region: rows above 500 for images taller than 600 pixels, and rows above 20 otherwise.

The resulting binary image is cleaned by removing small objects and small holes at a size threshold of 199 pixels, followed by two erosions and one dilation. The erosions detach tick marks that touch the waveform body and the dilation recovers part of the resulting loss. Contours are then extracted with `skimage.measure.find_contours`, and the returned bounds `Xmin`, `Xmax`, `Ymin` and `Ymax` are the extremes across *all* contours, that is the bounding box of every surviving object rather than of a single selected component.

Two practical consequences follow. First, because the bounding box spans all surviving contours, residual bright artefacts inside the search area widen the reported region. Second, the per-pixel Python loop makes this one of the slowest steps in the pipeline; it is a natural target for vectorisation.

5.2 Raster: `define_end_rois`

`define_end_rois` derives the two side regions in which axis ticks and labels are later sought. Given the coarse bounds and the image dimensions it returns two [`Xmin`, `Xmax`, `Ymin`, `Ymax`] lists:

Left ROI spans horizontally from column 0 to `Xmin - 1`, that is everything to the left of the waveform.

Right ROI spans horizontally from `Xmax` to the right-hand edge of the image.

Both regions share the same vertical extent. The top is set to `Ymin - 75` when `Ymin - 125 > 0`, and to row 1 otherwise; the bottom is the full image height. The upward extension is deliberate: axis labels are frequently printed slightly above the top of the traced waveform, so an ROI clipped exactly at `Ymin` would omit the highest label.

5.3 DICOM: region bounds from metadata

The DICOM branch does not call either function above. The waveform bounds are taken directly from the ultrasound region metadata as `RegionLocationMinX0`, `RegionLocationMaxX1`, `RegionLocationMinY0` and `RegionLocationMaxY1`, and the zero-velocity row is computed as `RegionLocationMinY0 + ReferencePixelY0`. The bounds are therefore exact rather than estimated, which removes a whole class of failure that the raster branch must tolerate.

6 Stage 3: Axis Calibration

Calibration establishes the mapping from image row to physical velocity. This is the stage where the two modalities differ most sharply: for DICOM it is a metadata lookup, whereas for raster images the entire scale must be recovered by locating tick marks and reading their printed values by OCR.

6.1 Raster: tick detection with `search_for_ticks`

`search_for_ticks` is called once per side with `side` set to "Left" or "Right", and returns twelve values. The important ones are `Cs` (tick contours), `CenPoints` (tick centres in ROI-local coordinates), `ROIAX` (the axis ROI cropped so that it contains the labels but not the tick strokes) and `TYLshift` (the offset used for that crop). The remainder are intermediate images and pass-through arguments consumed by `search_for_labels`.

The algorithm is:

1. Crop the side ROI produced by `define_end_rois` and threshold the greyscale image at an intensity of 127.
2. Find contours in the ROI and draw them into a working image.
3. **Right side only:** re-label the ROI and retain only components with an area between 5 and 200 pixels and a height below 5 pixels, that is objects shaped like short horizontal strokes. Columns are then blanked if they intersect more than 20 such objects or none at all. This suppresses the dense text of the adjacent panel, which is a problem on the right side but not the left.
4. Re-find contours on the filtered image and compute each contour's centre.
5. **Select the tick column.** For every column in the ROI, count the contours whose x extent includes that column, giving a one-dimensional profile. `scipy.signal.find_peaks` is applied to that profile with `height = 3`, requiring at least three aligned objects for a column to be considered.
6. Retain the contours lying on the selected column, recording their centres and endpoints.
7. **Left side only:** reject the worst endpoint outlier using a modified z -score against the median with $m = 8.0$.
8. Compute `TYLshift` and re-crop `ROIAX` so that OCR sees the numeric labels without the tick strokes, which would otherwise be read as hyphens.

Asymmetry in column selection. The two sides choose the tick column by different criteria, and this is the most consequential difference between them. For the right axis the strongest peak is chosen by the product of peak height and peak width:

```
peak_wid = signal.peak_widths(all, peaks)
maxID = np.argmax(vals["peak_heights"] * peak_wid[0])
```

For the left axis the selection is instead:

```
maxID = np.argmax(peaks)
```

Because `peaks` contains column indices in ascending order, taking its `argmax` selects the *last* entry, that is the right-most candidate column in the left ROI. This is the column nearest the waveform, which is where left-axis ticks are expected, so the behaviour is reasonable; but it is expressed as a maximum over positions rather than as an explicit choice of the last peak, which obscures the intent and makes the line easy to misread.

Failure when no ticks exist. Both selections index `peaks` unconditionally. On a scan with no ticks on a given side the profile has no peaks, `peaks` is empty, and the call raises `ValueError: attempt to get argmax of an empty sequence`. This is the normal, expected outcome for scans that print labels on one side only, and it is caught by the caller; see Section 6.7.

6.2 Raster: label detection and pairing with `search_for_labels`

This function reads the numeric tick values and associates each with a tick position. It returns the cropped ROI, the list of label strings (`number`), their paired positions in full-image coordinates (`positions`), and a full-size debug canvas.

Threshold sweep with a plausibility stop. Rather than committing to one binarisation, the function sweeps the threshold from 100 to 185 in steps of 5, running OCR at each value with

```
--psm 11 -c tesseract_char_whitelist=-0123456789
```

and stops as soon as every parsed number is divisible by 5. This exploits a strong prior about Doppler displays: velocity axes are labelled in multiples of five, so divisibility is an inexpensive proxy for a clean read. The whitelist restricts output to digits and the minus sign, and `psm 11` treats the crop as sparse text.

Box handling. Tesseract emits repeated rows per box, which are deduplicated by an index test; the first box is skipped, and empty or hyphen-only labels are discarded. Label centres are computed from the bounding boxes, with `TYLshift` added back on the right side to undo the earlier crop. If any two label centres lie within 20 pixels of each other, the code logs `"Characters too close"`, indicating that two labels may have been merged.

Pairing. Each label centre is matched to its nearest tick centre by Euclidean distance using `scipy.spatial.distance.cdist`, and the matched tick position is offset back into full-image coordinates. Finally `correct_number_format` strips a trailing hyphen from values such as `"40-"`, an artefact of a tick stroke that survived the crop.

Fallback. If pairing fails, the function re-runs OCR and uses the label box centres themselves as the tick positions. This is less accurate, because a label's centre is horizontally offset from its tick, but it keeps calibration possible when tick detection has partially failed. The `cdist` call is also the origin of the `ValueError: XA must be a 2-dimensional array` seen on scans with no labels on one side, since an empty list of centre points has no second dimension.

6.3 Raster: validation of ticks

`validate_axis_ticks` cleans a single side's ticks. It never mutates its inputs and returns a corrected copy. Values that cannot be parsed as floats are skipped, and if fewer than two ticks remain, or the value and position counts disagree, the inputs are returned unchanged.

Ticks are sorted by pixel row and then repeatedly filtered until stable:

- Ticks sharing an identical row are collapsed: if they carry the same value the last is kept, and if they conflict the whole cluster is dropped.
- The slope $\Delta\text{value}/\Delta y$ is computed between successive ticks. Slopes are clustered within a relative tolerance `spacing_tol = 0.2`, and the largest cluster is taken as dominant.
- A single anomalous slope at either end drops the corresponding end tick; exactly two consecutive anomalous slopes drop the tick between them; a specific three-slope pattern drops both an interior tick and an endpoint. Any more complex pattern only logs a warning and stops, on the principle that it is safer to keep suspect ticks than to guess.

One caveat worth recording: the function computes monotonicity flags for the ordered values but does not act on them. A non-monotonic set of tick values is therefore not rejected on that basis alone, and is only corrected if it also produces an anomalous slope pattern.

6.4 Raster: cross-checking the two axes

`validate_axis_pair` compares the sides where both were detected. It builds a value-to-row map for each side, intersects the tick values common to both, and warns if the mean row for a shared value differs by more than `y_tol_pixels = 5.0`. When the sides share no values the result is inconclusive and reported as agreement. The function is purely diagnostic: it returns a boolean and changes no data, and its result is used only to decide which side to trust during digitisation.

6.5 Raster: zero-line estimation

`estimate_zero_line_y_axis` finds the row corresponding to zero velocity for one side. If any tick value is zero within 10^{-3} it returns the mean row of those ticks. Otherwise, given at least two ticks, it fits a straight line of row against value with `numpy.polyfit` and returns the intercept at zero. The fit matters because it recovers the baseline even when it is off-screen or unlabelled, which is common when a display shows only positive velocities.

`estimate_zero_line_y` then fuses the two sides. With one side available it returns that estimate. With both, it prefers whichever side has at least three ticks when the other has fewer; if the sides are comparable it warns when they differ by more than 5 pixels and returns their mean. The result is passed to `segment_refinement`, where it decides waveform orientation (Section 7.3). A `None` result is tolerated and triggers the centroid-based fallback described there.

6.6 DICOM: calibration from metadata

The DICOM branch performs none of the above. `extract_dicom_metadata` reads the Sequence of Ultrasound Regions, tag (0018,6011), and selects the first region whose Region Spatial Format and Region Data Type are both 3, identifying the pulsed-wave spectral region. From that region it reads the bounding box, the reference pixel, the reference physical values and the physical deltas, listed in Table 3.

Only the header is read, using `stop_before_pixels=True`. Missing elements resolve to `None` rather than raising, and any exception causes the function to return an empty dictionary; in that case the absence of bounding-box keys will surface later as an arithmetic error during digitisation. The physical units tags (0018,6024) and (0018,6026) are recorded but not consulted during conversion, so the code implicitly assumes the conventional units of centimetres per second and seconds.

Key	Tag	Role
RegionLocationMinX0	(0018,6018)	ROI left
RegionLocationMinY0	(0018,601A)	ROI top
RegionLocationMaxX1	(0018,601C)	ROI right
RegionLocationMaxY1	(0018,601E)	ROI bottom
ReferencePixelX0	(0018,6020)	Origin column
ReferencePixelY0	(0018,6022)	Origin row (baseline)
ReferencePixelPhysicalValueX	(0018,6028)	Physical value at origin
ReferencePixelPhysicalValueY	(0018,602A)	Physical value at baseline
PhysicalDeltaX	(0018,602C)	Units per column
PhysicalDeltaY	(0018,602E)	Units per row

Table 3: DICOM tags used for calibration.

6.7 Tolerance of missing axes

Both entry points wrap each side’s tick and label search in its own `try/except`, initialising `Lnumber`, `Lpositions`, `Rnumber` and `Rpositions` to `None` beforehand. A side that fails leaves its variables as `None` and the pipeline proceeds on the other side. This is what allows scans that label only one axis to be processed at all, and it is by design rather than by accident.

The cost is diagnostic noise. In `segment_files.py` the handlers call `traceback.print_exc()` in addition to logging, so an entirely expected single-sided scan still prints full tracebacks for every image processed. The exceptions are handled, but they are not silenced, which makes a healthy batch run look as though it is failing.

7 Stage 4: Waveform Segmentation and the Three Tracing Methods

This is the algorithmic core of the package and it is shared by both modalities. It proceeds in two parts. First a single refined binary mask of the waveform body is produced. Then *three independent methods* extract a one-pixel-per-column trace of the waveform envelope. All three are computed for every scan; the choice between them is deferred to Section 10.

7.1 Orchestration: `segment_refinement`

`segment_refinement` is the single entry point for this stage and is called identically by both entry points, except for the `grow_forbid_yellow` flag which is `True` for raster images and `False` for DICOM. Its sequence is:

1. Build the refined binary mask with `refine_waveform_segmentation`.
2. Convert the image to greyscale and construct the permitted region `allowed` with `allowed_mask_from_roi`.
3. If `grow_forbid_yellow` is set, build a forbidden-region mask of instrument yellow with `hsv_yellow_tick_mask_bgr`.
4. Shrink the refined mask into a conservative seed with `_shrink_seed_for_region_grow`.
5. Grow that seed with `constrained_region_grow`.
6. Derive all three envelope traces with `compute_top_curve`.

It returns seven values: the refined mask, and a mask/coordinate pair for each of the morphological, ray-traced and grown traces. The latter two pairs are `None` if their method failed or produced an empty result, so all downstream code must tolerate missing traces.

7.2 The shared refined mask: `refine_waveform_segmentation`

This function produces the binary waveform body that both the morphological method and the region-growing seed are derived from. Unlike `initial_segmentation` it works on the greyscale OpenCV image and uses a fixed intensity threshold:

1. Convert BGR to greyscale and threshold at an intensity of 30.
2. Zero everything outside the region of interest. Note the asymmetric vertical margin: the top boundary is `Ymin - 50` rather than `Ymin`, deliberately admitting rows above the coarse bound so that systolic peaks clipped by Stage 2 are recoverable.
3. Remove small objects and holes (199 pixels), erode twice, dilate.
4. Dilate again, remove holes up to 999 pixels, apply a morphological closing, then a 3×3 median filter.
5. Label connected components and discard everything smaller than the second-largest area plus 10, which in practice retains only the main waveform body.
6. Blur with a Gaussian of $\sigma = 7$ and re-threshold at 0.5, smoothing the outline so that the envelope extracted from it is not dominated by pixel-level noise.

7.3 Envelope orientation: upper or lower

Doppler waveforms may be displayed above or below the baseline, so before any envelope is extracted the code must decide which side of the waveform body is the diagnostic envelope. This decision, held in the variable `keep` with value "upper" or "lower", is made once in `_morphological_top_curve_from_mask` and then *reused* for the ray tracer, ensuring that all methods trace the same side of the waveform.

When a zero line is available the mask outline rows are counted above and below it, and the waveform is treated as inverted when more of the outline lies below the baseline. Rows on the non-diagnostic side are then cleared. When no zero line is available the code falls back to the centroid row of the first labelled region and validates the choice with `check_inverted_curve`, which declares the trace inverted if the retained outline spans less than a fraction `tol = 0.25` of the waveform height; if so, the opposite side is kept instead.

7.4 Method 1: Morphological envelope

Implemented by `_morphological_top_curve_from_mask`. This is the original method and the only one guaranteed to be produced.

The mask is eroded once and the erosion subtracted from the original, `outline = mask - erosion(mask)`, leaving a one-pixel shell around the waveform body. The non-diagnostic side is cleared as described in Section 7.3, and `keep_one_pixel_per_column` reduces the shell to a single pixel per column, taking the visually uppermost or lowermost pixel according to `keep`.

The method is fast, deterministic and entirely dependent on the quality of the refined mask. Because the mask has been Gaussian-smoothed, the resulting trace is smooth but can be blunted at sharp systolic peaks, and any thresholding error in the mask is inherited directly.

7.5 Method 2: Ray tracing

Implemented by `ray_trace_waveform_segmentation` and used as the primary metric source downstream. Rather than deriving the envelope from a binary mask, this method re-examines the original image column by column, which allows it to follow a faint envelope that global thresholding would lose.

Its region of interest is computed by `ray_trace_roi_from_refined_mask` from the bounding box of the refined mask, padded by $\max(8, 3\%)$ of the box width horizontally, $\max(18, 35\%)$ of its height above and $\max(6, 6\%)$ below, then inset by 15 pixels horizontally. The generous top padding exists so that peaks excluded from the refined mask remain inside the search area. A further hard inset of 20 pixels per side is applied inside the tracer itself to exclude border artefacts.

The algorithm then proceeds as follows.

1. **Adaptive signal/background separation.** All ROI pixels are clustered in BGR colour space with k -means, $k = 3$. The cluster with the lowest mean greyscale is declared background and the remaining two clusters become signal candidates. Because the split is learned per image, this adapts to scans of differing brightness instead of relying on a fixed threshold.
2. **Instrument-yellow removal.** Pixels identified by `hsv_yellow_tick_mask.bgr` (hue 18–40, saturation > 90 , value > 90 , followed by an opening with a disk of radius 1) are

removed from the signal, so that the machine’s own yellow peak markers are not mistaken for the envelope.

3. **Continuity-constrained column scan.** Columns are traversed left to right. Each column is first searched only within $\pm \text{max_col_step_y}$ rows of the previous column’s pick, which enforces vertical continuity and prevents the trace jumping to an unrelated structure. Within that window, `_pick_y_from_column_signal` prefers the first run of three consecutive signal pixels, taking the uppermost or lowermost such run according to `keep`; if no run exists it falls back to the brightest pixel in the window. If the banded search fails entirely the whole column is searched and the result is clipped back into the permitted step.
4. **Gap filling and smoothing.** Columns with no pick are filled by linear interpolation, isolated outliers are suppressed by a Hampel filter (half-window 2, 3σ), and a width-3 median filter is applied. The narrow kernels are a deliberate compromise: they remove spikes while preserving the steep genuine slope into the diastolic foot.
5. The trace is rasterised, dilated by a disk of radius 1 and mapped back into full-image coordinates.

The step limit `max_col_step_y` defaults to $\max(30, \text{ROI height}/25)$ and is clamped to at least 3. It is the method’s most consequential parameter: too small and the trace cannot follow a steep systolic upstroke, too large and continuity no longer constrains it.

7.6 Method 3: Constrained region growing

Implemented by `constrained_region_grow`. This method exists to recover parts of the waveform that the fixed threshold in `refine_waveform_segmentation` misses, most notably the dimmer signal in the diastolic troughs, by expanding outward from a confident core rather than classifying each pixel independently.

Seed preparation. The refined mask is first shrunk by `_shrink_seed_for_region_grow`, which erodes it with a disk of radius `GROW_SEED_EROSION_RADIUS = 10` and intersects the result with the permitted region. Shrinking matters because the seed statistics drive the acceptance test: an over-thick seed biases the intensity model towards the bright waveform core and prevents growth into the troughs. If erosion removes every seed pixel the function logs a warning and falls back to the unshrunk intersection.

Constraints. Growth is restricted to `allowed`, the ROI mask produced by `allowed_mask_from_roi_bounds` (which applies the same $Y_{\min} - 50$ top margin as the refined mask). A separate `forbidden_mask` may additionally block expansion without excluding seed pixels; for raster images this carries the instrument-yellow mask, and for DICOM it is disabled via `grow_forbid_yellow=False` because yellow is rare there and masking it removes real signal.

Intensity model. The greyscale image is normalised to $[0, 1]$ over the whole image. From the seed the mean and standard deviation are computed, and an absolute floor is derived as

$$\text{min_intensity} = \max(\mu_{\text{seed}} - \max(5.0 \sigma_{\text{seed}}, 0.40), 0.05).$$

The initial acceptance band is $\mu_{\text{seed}} \pm 0.43$, clamped by that floor and by 1.

Growth. A breadth-first flood fill runs from every seed pixel. The neighbourhood is a Chebyshev ball of radius `GROW_NEIGHBOUR_CHEBYSHEV_RADIUS = 2`, that is a 5×5 neighbourhood, which lets the region bridge one-pixel gaps in a faint envelope; the `connectivity` argument only applies when the radius is 1. A candidate is rejected outright below `min_intensity`. Otherwise, because `update_seed_stats` defaults to `True`, the acceptance band is recomputed from the running mean and standard deviation of all accepted pixels as $\mu \pm \max(3.9\sigma, \text{intensity_tol})$, again clamped. This lets the region adapt as it moves into dimmer areas. Growth stops when the region reaches `max_area_growth = 3.0` times the seed area, which bounds both runtime and the extent of any leak into the background.

Envelope extraction. The grown mask is passed back through `morphological_top_curve_from_mask`, so Method 3 differs from Method 1 only in the mask it operates on, not in how the envelope is thinned.

7.7 Dispatch and comparison: `compute_top_curve`

`compute_top_curve` produces all three traces and returns them as three mask/coordinate pairs. The morphological envelope is always computed first because it establishes the shared `keep` orientation. The grown envelope is computed only if a non-empty grown mask was supplied, and the ray trace only if the original image and coarse x bounds were supplied; the ray call is wrapped in a `try/except` so a tracing failure degrades to `None` rather than aborting the scan.

	Morphological	Ray tracing	Region growing
Operates on	Refined binary mask	Original pixel data	Refined mask as seed
Separation	Fixed threshold of 30	Per-image k -means, $k = 3$	Adaptive intensity band
Envelope	Erosion difference, thinned	Per-column pick	Erosion difference, thinned
Continuity	Implicit in the mask	Enforced by step limit	Implicit in connectivity
Strength	Robust, deterministic	Follows faint envelopes	Recovers dim troughs
Weakness	Inherits threshold error; blunts peaks	Sensitive to step limit	Can leak; area capped
Always present	Yes	No	No

Table 4: The three tracing methods compared.

7.8 Debugging aids

Setting the module-level flag `SHOW_GROW_DEBUG_PLOTS`, or passing `grow_debug_plots=True` to `segment_refinement`, produces diagnostic figures of the region-growing pipeline and of the traced overlays. `ray_trace_waveform_segmentation` has an equivalent `debug_plots` argument showing the clustered signal mask, the yellow exclusion, the accepted three-pixel runs and the final trace. These are off by default because the batch path saves a fixed set of figures instead.

8 Stage 5: Digitisation and Calibration

Digitisation converts a set of image coordinates into a physical waveform. The two modalities use different functions for this, `plot_digitized_data_single_axis` for raster images and `plot_digitized_data_dicom` for DICOM, but both return the same seven-element tuple:

<code>Xplot, Yplot, Ynought, Xplot_o, Yplot_o, Xplot_g, Yplot_g</code>
--

where the unsuffixed pair is the primary series, `_o` the overlay and `_g` the region-grow series. `Ynought` carries the zero-velocity reference. Keeping the signature identical is what allows the two branches to reconverge for the remaining stages.

8.1 Condensing the curve to one sample per column

Both branches begin with the same reduction. A trace arrives as a list of (row, col) pairs which may contain several rows for one column. The pairs are swapped to (col, row), grouped by column with `pandas.groupby`, and the mean row per column is taken, after which the result is sorted by column. This guarantees that the digitised series is single-valued in time, which every downstream operation assumes.

8.2 Raster: choosing an axis and mapping rows to velocities

`plot_digitized_data_single_axis` calibrates from a single vertical axis. A side is usable if it carries at least two ticks and its value and position counts agree. The choice is made as follows: if only one side is usable, it is used; if both are usable, `validate_axis_pair` is consulted and the side with more ticks is selected either way, with a warning logged when the sides disagree. If neither side is usable the function returns empty series and logs an error. In other words tick count, not agreement, decides the outcome; agreement only affects logging. This is a deliberate choice to keep processing possible on partially labelled scans, and the disagreement warning is the signal to inspect such a scan manually.

Calibration itself is a two-point fit rather than a least-squares fit over all ticks. The (row, value) pairs are sorted by row and the line is taken through the extreme pair only:

$$a = \frac{v_{n-1} - v_0}{y_{n-1} - y_0}, \quad b = v_0 - a y_0, \quad \text{value} = a y + b.$$

Using the endpoints maximises the lever arm and so minimises the effect of quantisation on the slope, but it also means a single mislocated endpoint tick corrupts the entire scale. This is precisely the failure that `validate_axis_ticks` exists to prevent, which is why its endpoint-drop rules matter more than they might appear to.

Interior ticks are used for validation but not for the fit; the code proceeds only if there are at least two ticks and the first and last rows differ.

8.3 Raster: the arbitrary time axis

There is no equivalent of the vertical ticks for the horizontal axis, so the raster branch does not attempt to calibrate time at this point. Instead the column indices are normalised to the unit interval,

$$X_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1],$$

and the axis is labelled *arbitrary time scale*. The same $x_{\min}$ and $x_{\max}$ from the primary series are applied to the overlay and grow series, so all three share one time base and remain directly comparable.

Orientation correction. If the mean of the primary series is negative the sign of all three series is flipped. This handles displays that plot flow below the baseline, and applying it to all three series together preserves their alignment.

8.4 Raster: time-axis calibration from heart rate

Real time is recovered later, from the printed heart rate. The reasoning is that the mean interval between beat feet on the arbitrary axis must correspond to the true cardiac period 60/HR, giving a scale factor

$$s = \frac{60/\text{HR}}{T_{\text{arb}}}, \quad T_{\text{arb}} = \frac{x[f_{n-1}] - x[f_0]}{n_f - 1},$$

where f are the detected foot indices. `digitized_hr_scale_factor_for_raster` computes this from the ray series and the OCR dataframe, and `scale_raster_digitized_x` applies it to each series in turn. When the heart rate is missing or non-positive, or when foot spacing cannot be determined, the factor is 1.0 and the axis remains arbitrary.

This is the single most important dependency between stages: OCR failure on the heart rate line silently leaves the time axis uncalibrated. It also means the raster time axis inherits any error in the printed heart rate, which is why `refine_hr_from_local_roi` is worth its extra OCR pass.

8.5 DICOM: mapping pixels using physical deltas

The DICOM branch needs no fitting because the display geometry is recorded in the header. `_dicom_xy_from_curve_coords` maps each condensed point directly:

$$\begin{aligned} x_{\text{ref}} &= \text{RegionLocationMinX0} + \text{ReferencePixelX0}, \\ y_{\text{ref}} &= \text{RegionLocationMinY0} + \text{ReferencePixelY0}, \\ X_i &= \text{ReferencePixelPhysicalValueX} + (c_i - x_{\text{ref}}) \cdot \delta_x, \\ Y_i &= \text{ReferencePixelPhysicalValueY} + (r_i - y_{\text{ref}}) \cdot \delta_y. \end{aligned}$$

Here δ_x is `PhysicalDeltaX` and $\delta_y = -|\text{PhysicalDeltaY}|$. The reference pixel offsets default to zero, the reference physical values to zero and the deltas to one when absent. Crucially, the sign of δ_y is forced negative regardless of what the header states, because image rows increase downwards while velocity increases upwards. Consequently the DICOM branch has no need for the negative-mean inversion used on raster images, and does not apply it.

`Ynought` is set to `[ReferencePixelPhysicalValueY]`, defaulting to 0.0, whereas the raster branch always returns `[0.0]`.

The essential consequence is that DICOM X is in physical units immediately, with no dependence on OCR, on beat detection or on the printed heart rate. Table 5 summarises the difference.

	Raster	DICOM
Velocity axis	Two-point fit through OCR-read ticks	Reference pixel plus <code>PhysicalDeltaY</code>
Time axis	Normalised to $[0, 1]$, later scaled by heart rate	Reference pixel plus <code>PhysicalDeltaX</code> , physical at once
Requires OCR	Yes, for both ticks and heart rate	No
<code>Ynought</code>	<code>[0.0]</code>	<code>ReferencePixelPhysicalValueY</code>
Sign handling	Inverted if mean is negative	<code>PhysicalDeltaY</code> forced negative
Failure mode	No ticks, so no calibration at all	Missing tags, so arithmetic error

Table 5: Calibration compared across the two modalities.

8.6 Shape-preserving smoothing

Digitised traces retain single-pixel jitter, which would otherwise be amplified by the derivative-based foot detection of Section 9. `_smooth_1d_digitized_shape_preserving` addresses this in two steps: a Hampel filter with a half-window of 2 and a threshold of 3σ removes isolated spikes, then a Savitzky–Golay filter smooths the remainder. The window is 10% of the series length clamped to $[7, 21]$ and forced odd, with polynomial order 3 reduced if the window is too short. A rolling median is used as a fallback when the window would be smaller than five samples or the filter raises.

A Savitzky–Golay filter is chosen over a moving average because it fits a local polynomial and therefore preserves peak height and curvature, which is essential when the peak values themselves are the measurements of interest. Smoothing is applied only to the ray series and to the grow series, since the morphological trace is already smooth by construction.

8.7 Primary and overlay series: an asymmetry between entry points

Both digitisation functions accept a primary curve and an overlay, with a flag `overlay_is_ray` declaring which is which. The two entry points populate these arguments in opposite orders:

- `single_image_processing.py` passes the **morphological** trace as primary with `overlay_is_ray=True`.
- `segment_files.py` passes the **ray** trace as primary with `overlay_is_ray=False`.

Plot colours and legends are driven by the flag and so remain correct in both cases. Metrics are also unaffected on the raster path, because `plot_correction` receives each series explicitly. On the DICOM path, however, `waveform_metrics_from_digitized` treats its primary (`Xplot`, `Yplot`) argument as the ray series when labelling the output columns. The consequence is that for a DICOM file the column headed *Digitized Value (ray)* contains morphological metrics when the single-image entry point is used, and ray metrics when the batch entry point is used. The numbers are correct in both cases; only the attribution differs. This is worth resolving before the column labels are relied upon quantitatively.

9 Stage 6: Beat Detection and Haemodynamic Indices

With a calibrated waveform in hand, the pipeline identifies individual cardiac cycles and reduces them to the standard haemodynamic indices. The beat detection logic is shared between modalities and lives in `_beat_detection_pass_feet`; only the wrapper differs. `_waveform_peaks_troughs_values_from` is used when x is already physical, as on the DICOM path, and `_digitized_peaks_metrics_timescaled` when x is arbitrary and must be scaled by heart rate, as on the raster path.

9.1 Why feet rather than troughs delimit a beat

A beat could in principle be delimited by successive minima, but the diastolic minimum of a Doppler envelope is a broad, flat feature whose location is sensitive to noise. The *foot* — the onset of the systolic upstroke — is by contrast a sharp change in curvature and can be located far more repeatably. The pipeline therefore detects peaks first, uses them to anchor a search for feet, and only then finds the extrema within each foot-to-foot interval.

9.2 Peak proposal

Peaks are proposed with `scipy.signal.find_peaks` using two constraints derived from the signal itself rather than fixed thresholds:

- **Minimum separation.** A physiological upper bound of 200 bpm implies a minimum spacing of 0.3 s, converted to samples as $\max(1, \lfloor 0.3/\Delta x \rfloor)$ where Δx is the median sample spacing. Where x is still arbitrary, the cruder $\max(1, n/15)$ is used, which assumes no more than fifteen beats on screen.
- **Prominence.** Set to a quarter of the peak-to-peak amplitude within `_beat_detection_pass_feet`. Scaling with amplitude makes the criterion insensitive to the absolute velocity scale.

On the raster path this is done twice. The first pass runs on the arbitrary axis purely to obtain the mean foot spacing needed for the heart-rate scale factor; the second runs on the scaled axis with the physiological 0.3 s separation, and replaces the first result if it yields at least two feet and at least one peak and trough. The second pass cannot change the indices, since linear scaling of x does not move samples, but it does change the separation constraint expressed in samples and therefore which peaks survive.

9.3 Foot detection

`_detect_feet_indices_from_peaks` locates one foot per anchoring peak using the derivatives of the envelope. For each peak:

1. Define a backward search window covering the last `FOOT_SEARCH_FRACTION = 0.50` of the preceding peak-to-peak interval, tightened by 0.10 for all beats after the first, and ending `FOOT_MIN_SAMPLES_BEFORE_PEAK = 3` samples short of the peak. For the first peak the interval is estimated from the mean of the *other* peak intervals, which is more stable than the first observed gap; the beat is skipped if the window would extend before the start of the signal.
2. Smooth the window with a Savitzky–Golay filter of window at most `FOOT_DERIV_SMOOTH_WINDOW_MAX = 11` and order `FOOT_DERIV_SMOOTH_POLYORDER = 2`.
3. Resample onto a uniform grid in x before differentiating, so that irregular sample spacing cannot distort the gradients.

4. Take the maximum of the second derivative as the upstroke anchor, the point of greatest positive curvature.
5. Walk backwards from that anchor along the first derivative and accept the first low-slope point as the foot.

A height guard rejects candidates that sit too high on the upstroke:

$$y_{\text{foot}} \leq y_{\text{trough}} + \text{FOOT_MAX_REL_HEIGHT} (y_{\text{peak}} - y_{\text{trough}}), \quad \text{FOOT_MAX_REL_HEIGHT} = 0.55.$$

Feet are returned sorted and deduplicated. Beat detection is abandoned, with an informational log entry, if fewer than two feet are found, since a single foot delimits no beat.

9.4 Per-beat extrema

`_peaks_troughs_from_feet` treats beat i as the half-open interval $[f_i, f_{i+1})$ and extracts one peak and one trough from each. A rolling median of window 5 stabilises the selection. Where one of the originally proposed peaks falls inside the beat, the tallest such anchor is used in preference to the segment maximum, which keeps the reported peak consistent with the peak that defined the beat. The trough is then constrained to lie *after* the peak, reflecting the physiological ordering of systole and diastole, and only falls back to the beat-wide minimum if the post-peak segment is empty. Intervals of three samples or fewer are skipped.

9.5 Signal quality filtering

`_sqi_template_correlation_from_feet` implements a template-matching quality index. Each beat is resampled to 200 points, a median template is formed across all valid beats, and a beat is accepted if its Pearson correlation with the template is at least 0.85. Beats that are degenerate — shorter than three samples, or with standard deviation below 10^{-10} — are marked invalid. Two fallbacks prevent the filter from discarding everything: if no beat is valid, or if the template itself is flat, or if every beat is rejected, all beats are accepted.

This filter is disabled by default. `USE_SQI_FILTER` is `False` at `general_functions.py:276`, so every detected beat currently contributes to the reported metrics. The mechanism is complete and gated by a single flag, so enabling it is a one-line change, but the effect on reported values has not been characterised and it should be treated as experimental.

9.6 Index formulae

The indices themselves live in `hemodynamic_indices.py` as pure functions, deliberately separated from the image processing so that they can be tested and audited independently. Peak systolic and end-diastolic velocities are the means over all accepted beats,

$$PS = \text{mean}(y[\text{peaks}]), \quad ED = \text{mean}(y[\text{troughs}]),$$

with ED replaced by machine epsilon if it is exactly zero, guarding the divisions that follow. Let $\bar{v} = \text{mean}(y)$ over *all* envelope samples. Then:

$$\begin{aligned} S/D &= PS/ED && \text{resistive_index_from_ps_ed} \\ RI &= \frac{PS - ED}{PS} && \text{tamax_from_envelope_temporal_mean} \\ TA_{\text{max}} &= \bar{v} && \text{pulsatility_index_from_ps_ed_and_mean_velocity} \\ PI &= \frac{PS - ED}{\bar{v}} \end{aligned}$$

All are rounded to two decimal places, and each guards against a zero or non-finite denominator by returning 0.0.

Two points deserve emphasis. First, $TA_{\max}$ is the temporal mean of the whole digitised envelope, not of complete beats only, so a partial beat at either end of the trace biases both $TA_{\max}$ and PI ; the batch plots shade incomplete beats for exactly this reason. Second, `tamax_from_ps_ed_approximation`, which computes $(PS + 2ED)/3$, is a legacy surrogate used only by the OCR consistency checks of Section 4, where no envelope is available. It is not used on any digitised path, and the two must not be conflated when comparing digitised and OCR values.

Heart rate is not computed from the digitised trace. On the raster path it is an input, read by OCR and used to calibrate time; on the DICOM path it is neither needed nor reported.

10 Stage 7: Model Comparison and Selection

Stage 4 produces three candidate traces and the pipeline must return one. The criterion is agreement with the machine’s own printed measurements: whichever trace reproduces the OCR values most closely is assumed to have segmented the envelope most faithfully. This makes the OCR stage not merely a reporting convenience but the arbiter of segmentation quality, and it is why an **Fail** flag from Section 4 degrades the result even though the segmentation itself succeeded.

This stage applies to raster images only. The DICOM path has no independent ground truth and is discussed at the end of the section.

10.1 Mapping digitised values into the metric table

`plot_correction` inserts three empty columns into the OCR dataframe at positions 3 to 5, named *Digitized Value (ray)*, *Digitized Value (morph)* and *Digitized Value (grow)*, so that they sit beside the extracted **Value** column for visual comparison.

Metrics are then computed for each series — the ray series through `plot_correction` itself and the other two through `_digitized_peaks_metrics_timescaled`, which reuses the primary series’ arbitrary period so that all three share one time base. Each value is written into the row matched by `_df_word_metric_mask`, which performs a substring match on the **Word** column with a dedicated regular expression for ophthalmic labels. The metric words are PS, ED, S/D, RI, TA, PI in general, and PS, ED, PS/ED, RI, PI for ophthalmic scans, which report no time-averaged velocity.

The OCR **Value** column is never overwritten. It remains the reference against which the three digitised columns are scored.

10.2 Agreement scoring

`digitized_comparison.py` classifies each metric as a velocity or a ratio via `metric_kind_for_digitized_row`, because an absolute error of 0.05 means something very different for a velocity in centimetres per second than for a dimensionless index. Ratio metrics are those whose **Word** contains PS/ED, ED/PS, S/D, PI or RI; velocity metrics are those containing TA, PS or ED. The ratio tests are ordered before the bare PS and ED tests so that a compound label is not misclassified.

Each cell is then assigned a colour. Let t be the OCR value and p the digitised value. For velocities, comparison is on magnitudes, since either may carry a spurious sign:

$$AE = |p| - |t|, \quad PE = 100 \frac{AE}{|t|} \quad (|t| > 0).$$

Colour	Velocity rule	Ratio rule
Green	$AE \leq 5$ and $PE \leq 5\%$	$AE \leq 0.05$
Amber	otherwise	$0.05 < AE \leq 0.10$
Red	$AE > 10$ or $PE > 10\%$	$AE > 0.10$

Table 6: Traffic-light thresholds as implemented in `velocity_digitized_style` and `ratio_digitized_style`.

When $|t|$ is within 10^{-12} of zero the percentage error is undefined, and the cell is green only if $AE < 10^{-9}$ and red otherwise.

Discrepancy. The docstrings in `digitized_comparison.py` do not match the code. The module docstring and `velocity_digitized.style` state green at $AE \leq 5$ with $PE \leq 10\%$ and red at $AE > 15$ or $PE > 20\%$, whereas the implementation uses $PE \leq 5\%$ for green and $AE > 10$ or $PE > 10\%$ for red. The implemented rules are therefore stricter than documented. The values in Table 6 reflect the code. This should be reconciled, since the thresholds determine model selection and not merely cell colour.

10.3 Selecting the returned model

`traffic_light_counts.by_metric.kind` tallies each digitised column into two triples of green, amber and red counts, one for velocities and one for ratios, skipping any row whose OCR value or digitised value is missing, unparseable or non-finite. `select_best_digitized_model_for_image` then ranks the columns by the key

```
(velocity_greens, ratio_greens,
 -velocity_ambers, -ratio_ambers,
 -velocity_reds, -ratio_reds)
```

sorted in descending order. Velocity greens dominate because peak systolic and end-diastolic velocities are the primary clinical measurements and the ratios are derived from them; a trace that gets the velocities right will generally get the ratios right, whereas the converse does not hold. Ambers are penalised before reds so that a column with several near misses is preferred to one that is confidently wrong. Two fallbacks apply: if no column scored any cell, the first column holding any numeric value is taken; and if no candidate column exists at all, "ray" is returned.

`finalize_image_digitized_model_choice` applies the decision. It consults the selector, then requires the chosen series to have at least two points with matching x and y lengths; if not, it falls back through ray, morph and grow in that order until a usable series is found. It records the outcome in a **Returned model** column as `returned_model_cell_value`, for example "morph algorithm", and returns the selected (x, y) pair. When no dataframe is supplied, or the dataframe lacks the ray column, it defaults to ray without scoring.

The full raster ordering is therefore: digitise with `plot_digitized_data_single_axis`; populate the digitised columns with `plot_correction`; compute and apply the heart-rate scale factor to all three series; and finally select with `finalize_image_digitized_model_choice`.

10.4 DICOM: no independent ground truth

The DICOM path calls `waveform_metrics_from_digitized` instead of `plot_correction`, and neither `select_best_digitized_model_for_image` nor `finalize_image_digitized_model_choice` is invoked. There is no **Returned model** column, and the trace returned is simply the primary series supplied by the caller — which, per Section 8.7, differs between the two entry points.

A structural consequence deserves attention. Because DICOM files carry no printed metric panel in the extracted region, the **Value** column of the resulting dataframe is populated with the computed primary metrics rather than with an independent measurement. The HTML report nonetheless applies the same traffic-light styling, comparing each digitised column against **Value**. The primary column therefore always scores perfect green, since it is being compared with itself, and the other two columns are scored against the primary rather than against any ground truth. The colours remain useful as an indication of agreement *between* the three segmentation methods, but they must not be read as accuracy.

11 Batch Pipeline and Outputs

The batch entry point is `usseg.main.main`, which composes three stages, each wrapped in the profiling helper `prof` so that a `cProfile` report is written per stage:

```
usseg.setup_tesseract()
filenames = prof(usseg.get_likely_us, root_dir)
prof(usseg.segment, filenames)
prof(usseg.generate_html_from_pkl)
```

The stages communicate through pickle files rather than in memory, which means each can be re-run independently — a useful property when only the report needs regenerating after a long segmentation run.

11.1 File discovery: `get_likely_us`

`organise_files.get_likely_us` walks `root_dir` with `os.walk`, collecting files with extensions `.jpg`, `.png`, `.dcm` and `.dicom`, and tests each with `check_file_for_us`. The two modalities are triaged differently:

- **DICOM** files are accepted unconditionally. The container format is itself sufficient evidence that the file is an ultrasound.
- **JPG and PNG** files are passed to `scan_type_test`, which performs a cheap OCR probe and returns `Fail = 0` for a probable Doppler scan. This is necessary because a directory of exported images typically also contains B-mode frames, report pages and screenshots.

Accepted files are grouped into a dictionary keyed by a patient identifier scraped from the file path with a regular expression, and that dictionary is pickled to `likely_us_images`. Two details are worth recording. First, the identifier pattern differs by modality: `\d{4}` for DICOM and `\d{3}` for raster, so the same directory tree can yield inconsistent grouping. Second, the function returns a *flattened* list of paths, discarding the grouping, which is preserved only in the pickle. Downstream code therefore processes scans independently, with no per-patient aggregation.

An optional `use_parallel` flag routes discovery through a `ThreadPoolExecutor`. It defaults to `False` and `main` does not enable it. Since the probe is dominated by Tesseract calls, which release the interpreter lock, threading would be expected to help; the conservative default presumably reflects Tesseract's thread-safety being uncertain in this configuration.

11.2 Per-scan processing: `segment`

`segment_files.segment` accepts a list of paths, a single path, a dictionary, or a path to a pickle, normalising all of them to a list. Each scan is dispatched by extension to the DICOM or raster branch already described in Sections 5 to 10.

Output naming. Figures are written with the prefix `output_dir + f"{idx}_{base_name}"`. The loop index is included deliberately: DICOM exports frequently reuse basenames such as `001.dcm` across patient folders, and without the index later scans would silently overwrite earlier ones.

The parallel-list invariant. The function accumulates five lists — `scan_display_paths`, `Digitized_scans`, `Annotated_scans`, `Text_data` and `Mean_wave_scans` — which must remain index-aligned, because the report pairs them positionally with `zip`. Every failure branch therefore appends `None` rather than skipping, and this is the reason the exception handlers are

structured as they are. A branch that returned early without appending would silently shift every subsequent row of the report against the wrong scan.

Failure counting. `Fail` is incremented when digitisation raises. It is not aggregated into a run-level summary, so the count of failed scans must be recovered from the log or by counting `None` entries in the pickle. A per-run summary would be a worthwhile addition.

Memory hygiene. After each scan the code calls `plt.close("all")`, which is essential: the pipeline draws into figures 1 and 2 by number, and without closing them the axes of successive scans would accumulate. A subsequent loop attempts to delete named locals through `exec("del %s" % i)`. This has no effect in Python 3, because `exec` cannot bind or unbind local variables in a function scope; it is dead code and can be removed.

11.3 Rendered outputs

For each scan the batch path may write four PNG files into `output_dir`:

Suffix	Contents
<code>.Source.png</code>	DICOM only: the extracted Doppler frame, saved because a browser cannot display raw <code>.dcm</code> bytes
<code>.Annotated.png</code>	Figure 1: the original scan overlaid with the ROIs, detected ticks and the three traces
<code>.Digitized.png</code>	Figure 2: the calibrated waveforms with beat markers, saved at 900 dpi
<code>.MeanWaves.png</code>	Mean beat per method, via <code>save_mean_waves_ray_morph_grow_figure</code>

Table 7: Per-scan figures written by the batch pipeline.

Figure 2 is saved by explicit number, `plt.figure(2).savefig(...)`, because annotation may have left figure 1 current. The mean-wave figure is wrapped in its own handler and contributes `None` on failure, so a mean-beat problem cannot cost the scan its other outputs.

11.4 Serialised artifacts

`segment` pickles the five lists to `segmented.data` as a single list, in the order

```
[scan_display_paths, Digitized_scans, Annotated_scans,
Text_data, Mean_wave_scans]
```

and returns the same data to its caller. Passing `pickle_path=False` suppresses the write, which is what the single-image and test paths do. Note that `Text_data` holds live `pandas` dataframes, so the pickle is coupled to the `pandas` version used to write it.

11.5 HTML report

`visualisation.html.generate_html_from_pkl` reads the pickle and calls `generate_html`, writing `generated_segmented_data.html` into the working directory. It tolerates a four-element pickle from an earlier version by synthesising a list of `None` for the mean-wave column.

Each scan becomes one horizontally scrolling row of four images followed by its metric table, and every image is inlined as a base64 data URI so that the report is a single self-contained file. The trade-off is size: base64 inflates each image by roughly a third and nothing is shared between rows, so a report over a large batch becomes slow to open. Note also that the MIME

type is hard-coded to `image/png` even for JPEG inputs; browsers sniff the content and render it regardless, but the declaration is incorrect.

Table cells in the three digitised columns are coloured by `digitized_cell_background_style` using the rules of Table 6, with empty cells left white. The styling call is wrapped in a bare `except` that falls back to an unstyled cell, so a non-numeric value degrades the colour rather than the report. Column widths are estimated from the longest string in each column, which keeps the wide metric tables legible without a stylesheet.

12 Error Handling and Known Failure Modes

12.1 Degradation strategy

The pipeline is designed to degrade rather than to abort. Each stage is wrapped in its own handler, and a failure removes only the information that stage would have contributed. The rationale is that a batch of several hundred scans will always contain some that fall outside the assumptions of the layout, and a single such scan must not cost the operator the whole run.

The cascade of degradation is as follows:

Stage fails	Immediate effect	Downstream consequence
One axis side	That side's ticks are <code>None</code>	Calibration proceeds from the other side
Both axis sides	No calibration possible	Empty digitised series; no metrics
OCR heart rate	Scale factor is 1.0	Time axis stays arbitrary; velocities unaffected
OCR overall (Fail)	<code>plot_correction</code> skipped	No digitised columns, so model selection defaults to ray
One tracing method	Its series is empty	Selection falls back through ray, morph, grow
Digitisation	<code>None</code> appended to all output lists	Report row shows blanks but stays aligned
Mean-wave figure	<code>None</code> in that column only	Other three figures still written

Table 8: How failure at each stage propagates.

12.2 Commonly observed failures

Two exceptions dominate the console output of a healthy run and are worth recognising on sight, because both are expected rather than pathological.

`ValueError: attempt to get argmax of an empty sequence.` Raised inside `search_for_ticks` when the column profile for one side has no peaks, that is when the scan prints no ticks on that side. Newer ophthalmic scans label the right axis only, so this fires once per image for the left side. It is caught, and the pipeline calibrates from the right axis alone.

`ValueError: XA must be a 2-dimensional array.` Raised by `scipy.spatial.distance.cdist` inside `search_for_labels` when there are no label centres to pair, which follows from the same cause. Also caught.

Because `segment_files.py` calls `traceback.print_exc()` in these handlers as well as logging, both produce full tracebacks on stderr for every scan processed. The handling is correct but the reporting is misleading: a successful run looks like a failing one. Downgrading these two expected cases to a single-line debug message, while keeping full tracebacks for genuinely unexpected exceptions, would make the log usable as a monitoring tool.

`FutureWarning from binary_erosion.` `_shrink_seed_for_region_grow` calls `morphology.binary_erosion`, deprecated in scikit-image 0.26 and due for removal in 0.28. Behaviour is unchanged for now; the call should become `morphology.erosion` with the same footprint.

12.3 Limitations

- **Layout dependence.** Panel position, the yellow annotation colour and the tick-labelling convention are assumed throughout. The spatial pre-mask in `text_from_greyscale` and

the ROI fractions in `extract_dicom_label_text` are tuned to the machines in use and would need revisiting for another vendor.

- **Time calibration depends on OCR.** On raster images real time exists only if the printed heart rate is read correctly, and the pipeline cannot detect a plausible-but-wrong heart rate.
- **Single spectral region.** `extract_dicom_metadata` takes the first matching pulsed-wave region and `extract_doppler_from_dicom` handles only single-frame (H, W) and $(H, W, 3)$ arrays, raising on multiframe data.
- **No independent validation on DICOM.** With no printed metric panel, the traffic-light colours measure inter-method agreement rather than accuracy, and no model selection is performed.
- **Endpoint-only axis fit.** A single surviving bad endpoint tick rescales every velocity on the scan.
- **Partial beats bias means.** $TA_{\max}$ and PI average the whole envelope, including incomplete beats at either end.
- **Quality filtering inactive.** `USE_SQI_FILTER` is `False`, so no beat is excluded on quality grounds.

13 Implementation Notes

13.1 Performance considerations

Runtime is dominated by Tesseract. The threshold sweep in `search_for_labels` may invoke OCR up to eighteen times per axis, and the early stop on divisibility by five exists primarily to cut that cost. Whole-scan OCR runs twice more in `text_from_greyscale`, once through `image_to_data` for boxes and once through `image_to_string` for lines, plus a further constrained pass in `refine_hr_from_local_roi`.

`colour_extract_vectorized` replaced a per-pixel implementation with NumPy array arithmetic and is no longer a significant cost; `colour_extract` remains in the module and is unused. Discovery can be threaded through `use_parallel`, though this is off by default.

`main` wraps each stage in `prof`, writing a `cProfile` report named after the function, which is the intended starting point for any further optimisation.

13.2 Testing

Tests live in `tests/` with fixture images in `tests/resources`. Patient data is not committed, so `tests/single_image_processing_test.py` carries a placeholder path that must be pointed at a local scan; run directly, it also plots the extracted trace, which is the quickest way to sanity-check a change to the segmentation code.

13.3 Diagnostics and figure conventions

Figures are addressed by number: figure 1 is the annotated scan and figure 2 the digitised waveforms. Traces are coloured consistently throughout — morphological in blue (`#1f77b4`), ray tracing in red, and region growing in violet (`#b026ff`) — and the same convention is used in this document. Several functions accept a `debug_plots` argument that renders intermediate masks; these are off by default because the batch path saves a fixed set of figures instead.

14 Recommended Follow-Up

The following items were identified while documenting the pipeline. They are listed in rough order of how much they affect interpretation of the results.

1. **Reconcile the traffic-light thresholds with their docstrings** (Section 10). The implemented rules are stricter than documented and they drive model selection, so the discrepancy matters beyond presentation.
2. **Make the primary/overlay order consistent between entry points** (Section 8.7), so that the *Digitized Value (ray)* column means the same thing regardless of how a DICOM file was processed.
3. **Quieten the two expected axis exceptions** so that a clean run produces a clean log, while preserving tracebacks for unexpected failures.
4. **Replace** `morphology.binary_erosion` with `morphology.erosion` before `scikit-image` 0.28 removes it.
5. **Characterise and then enable the SQI filter**, or remove it, so that its status is unambiguous.
6. **Restrict $TA_{\max}$ and PI to complete beats**, since the incomplete-beat shading already identifies the samples to exclude.
7. **Enforce the monotonicity result in `validate_axis_ticks`**, which is currently computed and discarded.
8. **Unify the patient-identifier regular expressions in `check_file_for_us`**, presently four digits for DICOM and three for raster.
9. **Report a per-run summary** of processed, degraded and failed scans, which the parallel-list structure already makes easy to compute.
10. **Remove dead code**: the `exec("del ...")` loop in `segment`, the unused `colour_extract`, and the unread `patient_paths` configuration key.